

Relatório TP2

Dicionário Inteligente para Monitoramento de Fábrica

Isabela M. Furtado¹, Maria Eduarda M. Mota²

¹Instituto de Computação – Universidade Federal do Amazonas (UFAM)

Disciplina: Algoritmos e Estruturas de dados II

Turma: EC500 **Período:** 2024/2

Professor: Dr. Cesar Augusto Viana Melo

***Resumo.** Este relatório tem como objetivo comparar a eficiência de duas implementações de dicionários dinâmicos: uma baseada em tabela hash com tratamento de colisões por listas encadeadas e outra baseada em uma árvore AVL. A comparação será realizada com base em três critérios principais: o número de comparações realizadas nas operações de busca, inserção e remoção, o consumo de memória das implementações e sua escalabilidade com o aumento do número de elementos. Através dessa análise, buscamos entender as vantagens e limitações de cada abordagem, ajudando a determinar qual estrutura oferece a melhor performance em diferentes cenários de uso.*

1. Introdução

A eficiência das estruturas de dados é um fator crucial em muitas aplicações de software, especialmente quando lidamos com grandes volumes de informações que exigem operações rápidas de busca, inserção e remoção. Entre as várias opções disponíveis, os dicionários dinâmicos são amplamente utilizados devido à sua capacidade de armazenar pares chave-valor de forma eficiente. Neste contexto, duas implementações comuns de dicionários dinâmicos são a tabela hash e a árvore binária balanceada AVL.

A tabela hash, que usa uma função de dispersão para mapear chaves para índices em um array, pode ser altamente eficiente em termos de tempo de busca, inserção e remoção, especialmente quando a função de hash é bem projetada e as colisões são tratadas adequadamente, neste cenário o tempo médio esperado para a inserção, remoção e pesquisa é $\mathcal{O}(1)$ podendo chegar a $\Theta(n)$ no pior caso para busca. Um método clássico de resolução de colisões é a utilização de listas encadeadas, onde múltiplos elementos que mapeiam para o mesmo índice na tabela são armazenados em uma lista ligada.

Por outro lado, a árvore AVL, uma árvore binária de busca balanceada, mantém a ordem dos elementos e garante que as operações de busca, inserção e remoção sejam realizadas, no pior caso, em tempo $\mathcal{O}(\log n)$ onde n é o número de nós existentes na árvore, independentemente da ordem das inserções. Embora a árvore AVL possa ter um desempenho ligeiramente inferior em termos de tempo constante em comparação com a tabela hash, ela apresenta uma estrutura mais ordenada, o que pode ser vantajoso em certos cenários, como nas operações que requerem a busca por intervalos ou a manutenção de elementos em ordem.

2. Estrutura do Projeto

O projeto visa simular o sistema de uma fábrica que armazena e gerencia informações a respeito das bombas de pressão que nela operam.

2.1. Definição do Modelo de Dados:

Em um arquivo chamado *bomba.h* foram definidas as seguintes estruturas:

Tipo *t_bomba_pression* que é uma struct com um campo para a classe da bomba *Class_ID*, outros campos que guardam as medições de cada bomba e uma *flag* que indica se a bomba passou por manutenção durante seu período de funcionamento.

```
typedef struct bomba_s t_bomba_pression;

struct bomba_s{
    int class_id;
    double temperature;
    double vibration;
    double pressure;
    double flow_rate;
    double rpm;
    double operational_hours;
    int maintenance_flag;
};
```

Tipo *t_item_bomba* que é uma struct com um campo para o identificador da bomba *Pump_ID* e outro campo que é um ponteiro para uma lista encadeada que guardará elementos do tipo *t_bomba_pression*.

```
typedef struct item_bomba_s t_item_bomba;

struct item_bomba_s{
    int pump_id;
    t_lse *infos;
};
```

Tipo *t_resultado_funcao* que é uma struct com campos para guardar as medidas de temperatura, vibração e pressão de uma bomba.

```
typedef struct resultado_funcao t_resultado_funcao;

struct resultado_funcao{
    double temperature;
    double vibration;
    double pressure;
};
```

Estas estruturas são usadas pelas demais funções definidas neste mesmo arquivo header.

```

t_bomba_pression *criar_info_bomba(
    int class_id,
    double temperatura,
    double vibracao,
    double pressao,
    double fluxo,
    double rpm,
    double horas,
    int manutencao);

t_item_bomba *criar_bomba(int pump_id);

int comparar_bomba(void *bomba1, void *bomba2);

void inserir_bomba_avl(int pump_id,
    t_bomba_pression *bomba,
    t_ArvoreAvl *avl);

t_hash* inserir_bomba_hash(int pump_id,
    t_bomba_pression *bomba,
    t_hash *hash);

void imprimir_bomba(void *bomba);

void imprimir_bomba_file(void *boom, FILE *saida);

t_resultado_funcao *report_avl(int pump_id,
    t_ArvoreAvl *tree,
    char *report_funcao);

t_resultado_funcao *report_hash(int pump_id,
    t_hash *hash,
    char *report_funcao);

```

2.2. Implementação das Estruturas de Dados:

Dois dicionários foram implementados, cada um com uma estrutura distinta. As estruturas escolhidas foram:

- Árvore Binária de busca Balanceada (AVL)
- Tabela Hash com encadeamento

2.3. Desenvolvimento de Funcionalidades

Os dicionários foram projetados para realizar quatro operações básicas:

- Inserção (ADD)

A inserção em uma árvore AVL é projetada para ser rápida e manter o balanceamento, garantindo que a altura da árvore permaneça próxima de $O(\log n)$. O processo começa na raiz e segue o caminho determinado pela comparação das chaves, movendo-se para a esquerda ou direita até encontrar o local apropriado para inserir a nova chave. Após a inserção, a árvore verifica e ajusta o balanceamento de cada nó percorrido de volta até a raiz, aplicando rotações simples ou duplas, se necessário, para corrigir qualquer desbalanceamento. Esse rebalanceamento automático assegura que a AVL continue eficiente para buscas e outras operações, mesmo com a adição frequente de novos elementos, mantendo a estrutura ideal para dicionários que demandam acesso e inserções rápidos.

A inserção em uma tabela hash com encadeamento funciona calculando o índice do elemento com uma função hash; caso o índice já esteja ocupado (colisão), o novo elemento é adicionado à lista encadeada associada a esse índice. Essa lista, chamada de "encadeamento separado", armazena todos os elementos que compartilham o mesmo índice, inserindo o novo elemento ao final dela. Essa abordagem permite que múltiplos elementos ocupem o mesmo espaço na tabela sem sobrescrever dados, mantendo a complexidade de tempo média da inserção em $O(1)$ e garantindo eficiência mesmo em situações com colisões.

- Busca (SEARCH)

A busca em uma árvore AVL para um dicionário é eficiente, pois aproveita o balanceamento automático da estrutura para garantir acesso logarítmico. Iniciando pela raiz, o algoritmo compara a chave desejada com as chaves dos nós, movendo-se à esquerda ou à direita até encontrar a chave ou chegar a um ponto onde ela não está presente. Como a AVL mantém sua altura mínima ao realizar rotações em cada inserção e remoção, a profundidade é controlada, o que evita a degeneração em uma lista e assegura que o tempo de busca seja rápido e consistente, mesmo para grandes volumes de dados.

A função de busca em uma tabela hash com encadeamento segue o processo de calcular o índice usando a função hash aplicada à chave do elemento. Se houver colisões (ou seja, múltiplos elementos no mesmo índice), a busca percorre a lista encadeada associada ao índice até encontrar o elemento com a chave desejada ou até o final da lista. Se o elemento for encontrado, a função retorna o valor associado à chave, caso contrário, retorna 'NULL', indicando que a chave não está presente na tabela. Esse processo garante que a busca seja eficiente, com um tempo médio de $O(1)$, embora possa degradar para $O(n)$ em casos de muitas colisões no mesmo bucket.

- Remoção (REMOVE)

A remoção de um elemento em uma árvore AVL envolve três principais etapas: encontrar o nó, removê-lo, e então balancear a árvore, se necessário. A busca pelo nó é feita com a mesma lógica da inserção, ou seja, comparando a chave do nó atual com a chave procurada e seguindo para o filho esquerdo ou direito. Após a remoção, a árvore pode ficar desbalanceada, então rotações são aplicadas para garantir que a árvore continue balanceada.

A remoção em uma tabela hash com encadeamento envolve três passos principais: calcular o índice usando a função hash, procurar o elemento na lista encadeada associada a esse índice, e remover todo conteúdo associado àquele identifica-

dor, ou seja, as medições guardadas nas listas.

- Gerar Relatório (REPORT) Em ambas as estruturas a geração de relatório é feita de maneiras semelhantes, a diferença é que as funções de buscas são para os devidos fins de cada código, nesse (REPORT) temos o "MEAN" onde é retornado média das medições de uma bomba, o "MIN" retornado os valores mínimos associados à bomba e o "MAX" valores máximos da bomba, para isso é necessário percorrer as listas com informações e fazer comparações, para só assim o resultado ser retornado.

3. Processo de Instrumentação

Para instrumentar o código das estruturas de dados, foram adicionados trechos de código responsáveis por contabilizar as comparações realizadas durante as operações de inserção, remoção, busca e redimensionamento. O objetivo foi obter informações precisas sobre a quantidade de comparações realizadas em cada operação, possibilitando uma análise detalhada do desempenho das estruturas.

4. Análise de Desempenho:

Teste	Nome do Arquivo	Quantidade de Amostras
1	dataset_01_perc.in	34
2	dataset_05_perc.in	234
3	dataset_10_perc.in	484

Tabela 1. Tamanho das entradas

4.1. AVL

Teste	Nome do Arquivo	Inserções	Buscas	Remoções
1	dataset_01_perc.in	76	30	32
2	dataset_05_perc.in	442	322	351
3	dataset_10_perc.in	955	733	772

Tabela 2. Quantidade de comparações realizadas nas operações de Inserção, Busca e Remoção

Tomando a coluna *Quantidade de Amostras* da Tabela 1 e as três últimas colunas da Tabela 2, podemos construir o seguinte gráfico:

Na Figura 1 observamos que de a operação de maior custo na AVL é a inserção, isto pode ser consequência de inserções de dados que estão parcialmente ordenados, assim causando desbalanceamentos na árvore.

4.2. Tabela Hash com encadeamento

Tomando a coluna *Quantidade de Amostras* da Tabela 1 e as cinco últimas colunas da Tabela 3, podemos construir o seguinte gráfico:

Na Figura 2 observamos que de a operação de maior custo na Hash é a inserção, provavelmente por conta de colisões.

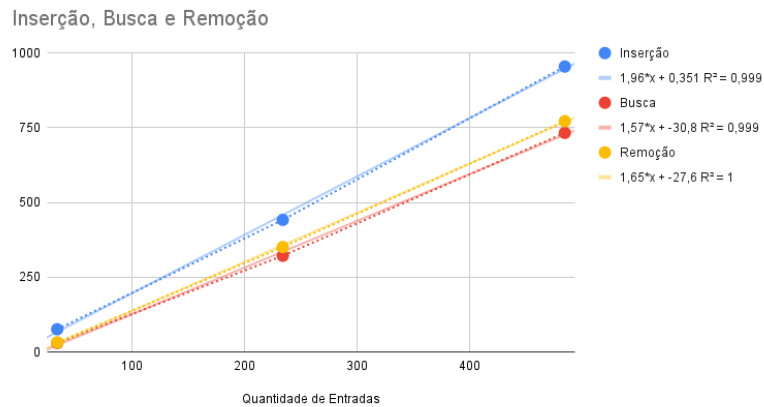


Figura 1. Este gráfico relaciona a quantidade de amostras presentes nos arquivos de entrada com o número de comparações realizadas nas operações de Inserção, Busca e Remoção na AVL

Teste	Nome do Arquivo	Inserções	Buscas	Remoções	Colisões	Movimentações
1	dataset_01_perc.in	9	8	6	6	5
2	dataset_05_perc.in	139	58	92	29	22
3	dataset_10_perc.in	271	73	72	40	65

Tabela 3. Quantidade de comparações realizadas nas operações de Inserção, Busca e Remoção e contagem das colisões e movimentações na Hash

4.2.1. AVL X Tabela Hash com Encadeamento

Ao analisar o gráfico da Figura 3 percebemos que as curvas das operações de Inserção, Busca e Remoção na AVL são sempre maiores que as curvas das operações de Inserção, Busca e Remoção na Hash. Portanto concluímos que uma Tabela Hash com Encadeamento quando bem implementada tem custo menor do que uma Árvore AVL.

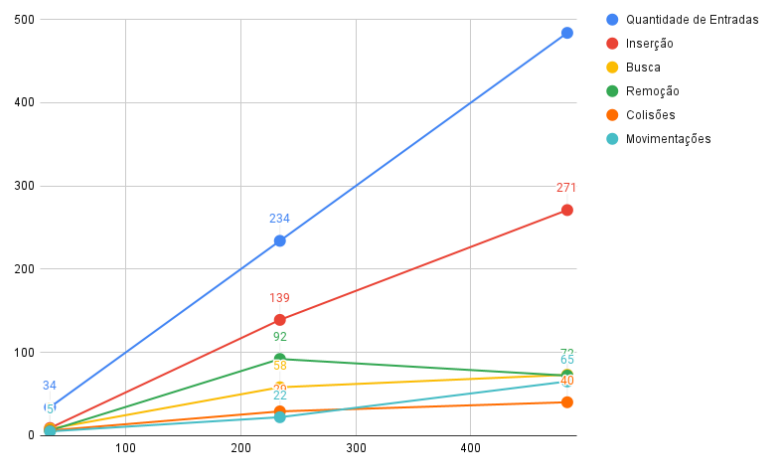


Figura 2. Este gráfico relaciona a quantidade de amostras presentes nos arquivos de entrada com o número de comparações realizadas nas operações de Inserção, Busca e Remoção na Hash

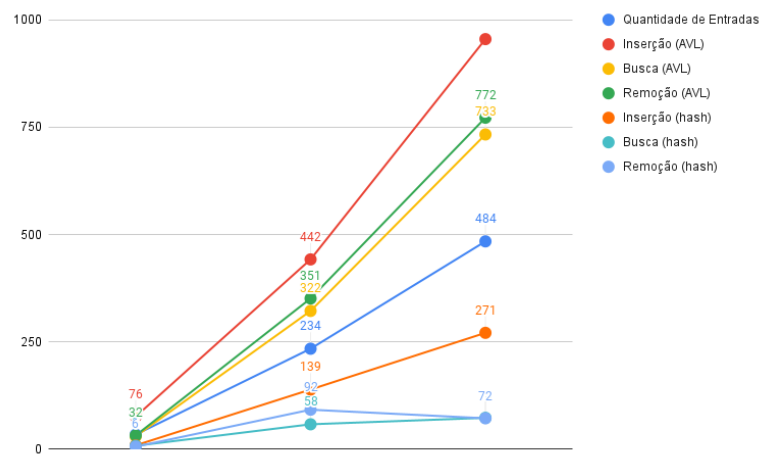


Figura 3. Este gráfico mostra as curvas das operações de Inserção, Busca e Remoção na AVL e na Hash